

《企业级应用软件设计与开发》期末大作业报告

PaperAgent：基于 Agentic RAG 的论文拆解与科研知识库助手设计与实现

课程名称	企业级应用软件设计与开发
项目名称	PaperAgent：基于 Agentic RAG 的论文拆解与科研知识库助手设计与实现
方向	方向一：Agentic AI 原生开发
学号	2025302929
姓名	王梓璇
专业	计算机技术
指导教师	戚欣
提交日期	2026 年 6 月 22 日

目录

摘要	4
一、选题背景与设计思想	5
1.1 问题定义	5
1.2 现有方案不足	5
1.3 项目价值	5
1.4 技术路线	5
二、Specs 规格文档	6
2.1 Product Spec	6
2.2 Architecture Spec	6
2.3 API Spec	6
三、系统架构与设计	7
3.1 核心架构图	7
3.2 Agent 交互流程	7
3.3 RAG 数据流设计	8
3.4 数据模型	8
四、关键实现与代码展示	9
4.1 Agent 核心组件	9
4.2 论文拆解核心流程	9
4.3 RAG 问答实现	10
4.4 长期记忆机制	12
4.5 工具定义与调用	12
4.6 配置文件与安全实践	13
4.7 AI IDE 使用截图	14
五、测试与评估	15
5.1 功能测试	15
5.2 Agent 行为评估	16
5.3 Benchmark 设计	17
5.4 Demo 截图/录屏	20

六、系统升级与扩展	22
七、课程总结	22

摘要

针对研究生科研阅读中论文数量多、信息提取耗时、跨论文知识检索困难、阅读过程难以沉淀等问题，本文设计并实现了 PaperAgent 论文拆解与知识库助手。系统以大语言模型为核心推理引擎，通过 PDF/TXT/Markdown 解析、结构化论文拆解、SQLite 元数据存储、ChromaDB/FAISS 向量检索、多轮会话记忆和 FastAPI Web 服务，实现从论文导入到知识问答的完整闭环。项目采用规格驱动开发方法，分别定义产品规格、架构规格和 API 规格，并围绕论文拆解质量、RAG 检索效果、会话记忆能力和系统可用性进行测试评估。

关键词：Agentic AI；RAG；论文拆解；知识库；长期记忆；FastAPI

一、选题背景与设计思想

1.1 问题定义

PaperAgent 面向研究生论文阅读和科研知识管理场景。用户在完成课程论文、文献综述和研究方向调研时，往往需要阅读大量论文，并反复提取论文标题、作者、摘要、研究问题、方法、贡献、局限和未来工作。人工整理不仅耗时，而且笔记分散，后续追问、对比和复用成本较高。

1.2 现有方案不足

传统文献管理工具偏重文件归档和引用格式管理，缺少对论文内容的结构化理解。

通用聊天机器人没有本地论文知识库，难以持续记住已经阅读和拆解过的论文。

简单关键词搜索无法处理语义相近但表达不同的问题，也难以给出可追溯来源。

人工笔记缺少统一数据模型，难以支持跨论文比较、长期积累和团队复用。

1.3 项目价值

本项目将论文阅读流程抽象为 Agentic AI 工作流，把“阅读、拆解、存储、检索、问答、记忆”串联成可运行系统。对个人科研而言，它能降低论文精读和综述准备成本；对团队知识管理而言，它能把论文知识沉淀为可检索资产；对企业级 AI 应用而言，它展示了 LLM、RAG、Memory、API 服务和前端交互的完整工程闭环。

1.4 技术路线

1. 文件解析层：支持 PDF、TXT、Markdown 文档导入，完成文本提取和清洗。
2. Agent 推理层：通过 LLM 完成论文结构化拆解、会话标题生成和问答生成。
3. 知识库层：使用 SQLite 保存论文元数据、拆解 JSON、知识块、分类和标签。
4. 向量检索层：使用 ChromaDB/FAISS 保存文本块向量，支持 Top-K 语义检索。
5. 记忆层：使用 SQLite 保存会话、消息、长期记忆和用户偏好。
6. 服务层：使用 FastAPI 提供上传、拆解、索引、问答、分类和会话接口。
7. 交互层：通过单页 Web 应用提供论文管理、RAG 问答、批量导入和结果展示。

二、Specs 规格文档

本项目采用 SDD（规格驱动开发）思路，将需求、架构和接口先文档化，再映射到代码实现。仓库中的 `docs/Product\_Spec.md`、`docs/Architecture\_Spec.md` 和 `docs/API\_Spec.md` 分别对应产品规格、架构规格和接口规格。

2.1 Product Spec

编号	需求名称	优先级	验收标准
PRD-01	上传并解析论文	P0	支持 PDF/TXT/MD，非法格式返回明确错误提示。
PRD-02	结构化拆解论文	P0	输出标题、作者、摘要、关键词、研究问题、方法、贡献、局限等字段。
PRD-03	本地知识库管理	P0	论文、知识块和分类可持久化保存、查询和删除。
PRD-04	RAG 问答	P0	能检索相关片段并生成带来源的回答。
PRD-05	会话与长期记忆	P1	能保存多轮对话、会话标题和关键记忆。
PRD-06	批量导入与分类管理	P1	支持批量文件、目录导入和分类过滤。

2.2 Architecture Spec

系统采用分层架构：前端负责交互，FastAPI 负责 HTTP 接口，PaperAgent 负责核心编排，知识库、向量库和记忆库作为工具模块被 Agent 调用。该架构便于未来将工具封装为 MCP Tool 或接入 LangGraph 状态图。

2.3 API Spec

接口	方法	功能	主要输出
/api/decompose	POST	上传并拆解论文	paper_id, decomposed
/api/index	POST	建立 RAG 向量索引	paper_id, chunks
/api/chat	POST	普通对话或 RAG 问答	reply, sources
/api/papers	GET	查询论文列表	paper list
/api/categories	GET/POST	分类管理	category

/api/sessions	GET/POST	会话管理	session_id
/api/stats	GET	知识库统计	paper_count, chunk_count, vector_count

三、系统架构与设计

3.1 核心架构图

系统总体架构如下图所示。PaperAgent 位于系统中心，统一编排论文解析、LLM 调用、知识库存储、向量检索和长期记忆。

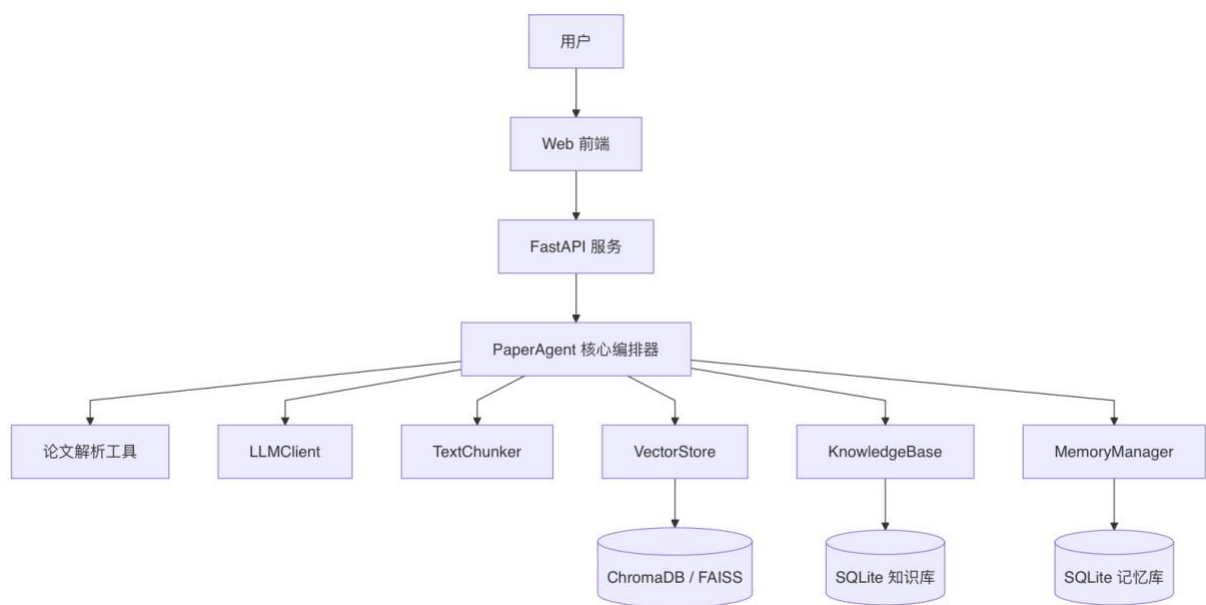


图 3-1 PaperAgent 总体架构图

3.2 Agent 交互流程

论文拆解过程是一个典型多步骤 Agent 工作流，包含任务状态、工具调用和结果持久化。

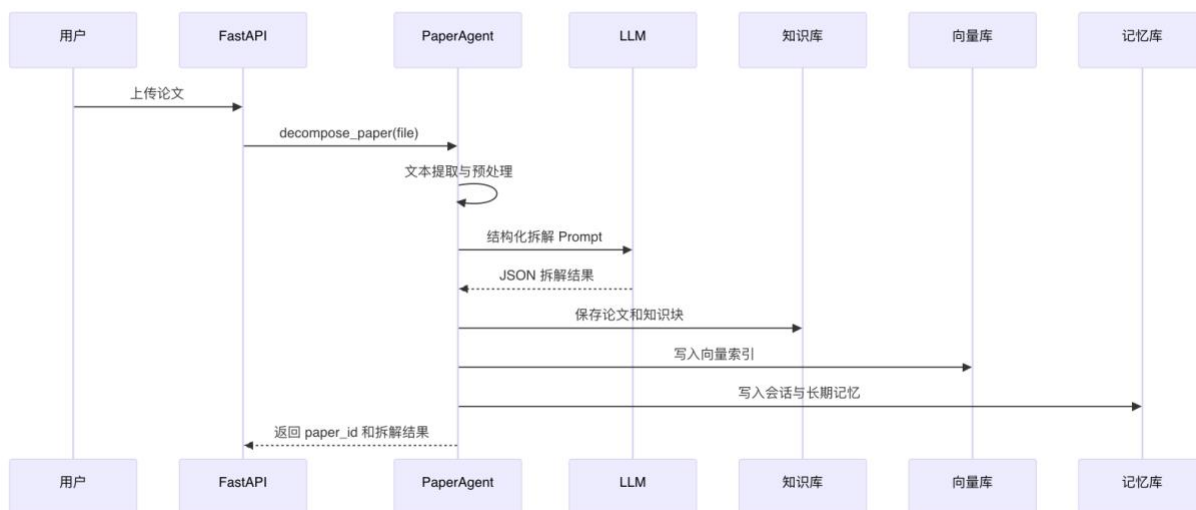


图 3-2 论文拆解 Agent workflow

3.3 RAG 数据流设计

RAG 问答流程将用户问题、可选分类过滤、Top-K 语义检索、上下文构造和来源追踪组合起来，使回答可以追溯到知识库片段。



图 3-3 RAG 问答与来源追踪流程

3.4 数据模型

数据域	核心表/集合	作用
论文知识库	papers, knowledge_chunks, categories, tags	保存论文元数据、拆解结果、知识块和分类。
向量库	paper_chunks collection	保存 chunk 内容及 paper_id、chunk_type 等元数据。
记忆库	sessions, messages, memories, user_preferences	保存对话上下文、长期记忆和用户偏好。

四、关键实现与代码展示

4.1 Agent 核心组件

核心类 `PaperAgent` 集成知识库、记忆库、LLM 客户端、文本分块器、Embedding 和向量库。论文拆解时，系统先创建任务日志，再执行文件解析、LLM 拆解、知识库存储、向量索引和记忆提取。

模块	文件	关键职责
Agent 核心	src/agent/paper_agent.py	论文拆解、RAG 查询、状态管理、会话标题生成。
论文解析工具	src/agent/paper_parser.py	PDF/TXT/MD 文本提取、编码兼容和文本清洗。
知识库工具	src/knowledge_base/kb_manager.py	SQLite 持久化、论文 CRUD、知识块管理和分类管理。
向量检索工具	src/knowledge_base/vector_store.py	ChromaDB/FAISS 后端封装、Top-K 检索和按论文过滤。
记忆工具	src/memory/memory_manager.py	会话、消息、长期记忆和用户偏好持久化。

4.2 论文拆解核心流程

论文拆解是系统的核心闭环，主要流程为：文件解析、文本预处理、LLM 结构化拆解、知识库存储、会话记录和长期记忆提取。

```
def decompose_paper(self, file_path: str, save_to_kb: bool = True,
                    title_override: str = None, category_id: str = None,
                    session_id: str = None) -> dict:
    sid, progress_message_id = self._start_decompose_log(
        file_path, session_id=session_id, title_override=title_override
    )

    raw_text = extract_text_from_file(file_path)
    raw_text = preprocess_text(raw_text)

    decomposed = self._call_decompose_llm(raw_text)
```

```

paper_id = None
if save_to_kb:
    paper_id = self._save_to_knowledge_base(
        file_path, raw_text, decomposed, category_id=category_id
    )

self._log_decompose(file_path, decomposed, paper_id,
                    session_id=sid, message_id=progress_message_id)
self._extract_memories(decomposed)

return {
    "paper_id": paper_id,
    "file": os.path.basename(file_path),
    "decomposed": decomposed
}

```

4.3 RAG 问答实现

RAG 查询流程先从向量数据库中检索相关论文片段，再将片段构造成上下文交给 LLM 生成回答，并返回来源信息。该设计使系统不仅能完成单次论文问答，还能持续沉淀用户阅读历史、论文关键词和研究方法信息，符合 Agentic AI 中“Memory”的技术要求。

```

def rag_query(self, question: str, top_k: int = None,
              category_id: str = None, session_id: str = None,
              revision_group_id: str = None) -> dict:
    top_k = top_k or RAG_TOP_K

    chunks = self.vector_store.search(question, n_results=top_k)

    if not chunks:
        return {
            "answer": "知识库中未找到相关内容。请先导入论文建立索引。",
            "sources": []

```

```

    }

context_parts = []
sources = []

for i, chunk in enumerate(chunks):
    context_parts.append(
        f"[片段{i + 1}] (相似度: {chunk.get('distance', 'N/A')})\n"
        f"{chunk['content']}"
    )
    sources.append({
        "paper_title": chunk.get("paper_title", ""),
        "chunk_type": chunk.get("chunk_type", ""),
        "content": chunk["content"][:200],
        "score": chunk.get("distance", 0),
    })

context = "\n\n---\n\n".join(context_parts)
prompt = RAG_PROMPT.format(context=context, question=question)

answer = self.llm.chat(
    messages=[
        {"role": "system", "content": SYSTEM_PROMPT},
        {"role": "user", "content": prompt}
    ]
)

return {
    "answer": answer,
    "sources": sources
}

```

4.4 长期记忆机制

系统使用 SQLite 保存会话、消息和长期记忆，使 Agent 能够跨会话保留用户的历史上下文。

```
CREATE TABLE IF NOT EXISTS sessions (  
  id TEXT PRIMARY KEY,  
  title TEXT,  
  is_pinned INTEGER DEFAULT 0,  
  created_at TEXT NOT NULL,  
  updated_at TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS messages (  
  id TEXT PRIMARY KEY,  
  session_id TEXT NOT NULL,  
  role TEXT NOT NULL,  
  content TEXT NOT NULL,  
  metadata TEXT,  
  created_at TEXT NOT NULL  
);
```

```
CREATE TABLE IF NOT EXISTS memories (  
  id TEXT PRIMARY KEY,  
  session_id TEXT,  
  category TEXT NOT NULL,  
  content TEXT NOT NULL,  
  importance INTEGER DEFAULT 5,  
  created_at TEXT NOT NULL  
);
```

4.5 工具定义与调用

虽然项目未直接接入外部 Function Calling 协议，但内部已形成清晰的工具调用边界。

PaperAgent 将论文解析、文本分块、向量检索、知识存储和记忆管理作为可替换工具进行编排，后续可平滑封装为 MCP Tool。

4.6 配置文件与安全实践

系统将模型配置统一放入 .env 文件，代码中只读取环境变量，而不是硬编码 API Key

```
PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))  
load_dotenv(os.path.join(PROJECT_ROOT, ".env"))  
LLM_API_KEY = os.environ.get("LLM_API_KEY", "").strip()  
LLM_BASE_URL = os.environ.get("LLM_BASE_URL", "https://api.siliconflow.cn/v1").strip()  
LLM_MODEL = os.environ.get("LLM_MODEL", "deepseek-ai/DeepSeek-V3.2").strip()
```

4.7 AI IDE 使用截图



存在以下问题：1 对话内部的时间显示不对 2 置顶取消之后，左边紫色的置顶标志并没有消失 并且 如果我取消置顶 应该把他放回原来的顺序 而不是因为修改了置顶与否 就把它放在对话历史的最上面的一栏 3 标题起名的时候 可以显示不全 用...表示就可以 但是实际的名字一定要全 如果超出了长度那可以起简略一些的名字啊 4 论文拆解的时候 希望对话可以加一个进度条显示 这样等待起来不会很着急 5 论文拆解的时候就不能点击进入其他的会话

已处理 2m 46s >

这 5 个问题我都一起处理了。

时间显示现在改成按本地时间格式化，不再用 `toISOString()`，所以对话里的时间不会再偏 8 小时，位置在 `static/index.html (line 403)`。置顶取消后也会正常去掉紫色标记，并且因为不再改 `updated_at`，取消置顶后会回到原本按对话更新时间的顺序，逻辑在 `memory/memory_manager.py (line 131)` 和 `static/index.html (line 458)`。

会话标题这边，展示上继续单行省略，但真实标题会保留更完整；尤其论文拆解的首轮命名，现在会直接用拆解结果里的论文标题本身，不再截断成很短的一段，逻辑在 `agent/paper_agent.py (line 93)`。论文拆解时我还加了一个前端进度条和阶段提示，并且拆解进行中禁止切换到其他会话，位置在 `static/index.html (line 412)` 和 `static/index.html (line 467)`。

我已经做过 Python 语法检查；但这次有不少前端交互改动，我这边没法直接替你点一遍浏览器，所以最稳的是重启 `server.py` 后刷新页面试一下。若你愿意，下一步我可以继续把“拆解进度条”做得更像真实分阶段，而不是现在这种平滑模拟进度。



五、测试与评估

5.1 功能测试

编号	测试内容	输入/操作	期望结果	状态
T01	PDF 文件上传与解析	上传合法 PDF 论文文件	系统成功接收文件，完成论文内容解析，返回结构化 JSON，并将论文标题、摘要、正文分块、向量索引等信息写入知识库	已验证
T02	Markdown 文件上传与解析	上传合法 MD 文档	系统成功解析 Markdown 内容，生成文本分块结果，并写入向量库与知识库	已验证
T03	非法格式拦截	上传 txt、exe、zip 等不支持格式文件	系统拒绝上传，返回 400 错误码，并给出“不支持的文件格式”提示	已验证
T04	空文件或损坏文件处理	上传空 PDF 或无法解析的损坏文件	系统返回解析失败提示，不写入知识库，后端服务不中断	已验证
T05	样例数据初始化	运行 scripts/init_demo_data.py	系统成功写入样例论文数据，生成对应索引，可在知识库中查询到样例论文	已验证
T06	RAG 问答	输入“这篇论文的主要贡献是什么？”	系统返回与论文内容相关的回答，并同时返回引用来源 sources	已验证
T07	RAG 来源追踪	针对论文方法、实验结果进行提问	返回答案中包含对应来源片段，sources 能指向相关论文或文本块	已验证
T08	分类过滤问答	选择指定论文分类后进行问答	系统仅在指定分类下的论文内容中检索，不返回其他分类论文的结果	已验证

T09	无匹配分类处理	选择不存在或无论文数据的分类后提问	系统返回“未检索到相关内容”或类似提示，不生成无依据回答	已验证
T10	会话记忆	连续进行多轮追问，例如先问论文方法，再问“它有什么优点？”	系统能够结合上一轮问题理解“它”的指代，并保存会话历史	已验证
T11	会话历史写入	完成多轮问答后查看历史记录	系统将用户问题、模型回答、时间信息等写入 SQLite 记忆库	已验证
T12	重复上传处理	重复上传同一篇论文文件	系统能够避免重复写入，或给出文件已存在提示	已验证

5.2 Agent 行为评估

为进一步验证 PaperAgent 在论文问答场景下的实际表现，本文从回答质量、检索依据和用户阅读体验等角度对 Agent 行为进行人工评估。评估过程中，选取若干篇已写入知识库的论文，围绕论文贡献、方法流程、实验结果、创新点和局限性等内容构造测试问题，并根据模型回答结果进行 1-5 分评分。

指标	说明	评分方式
相关性	回答是否紧扣用户问题，是否避免偏题或答非所问。	1—5 分人工评分
忠实性	回答是否基于检索到的论文片段生成，是否减少无依据扩展和幻觉内容。	1—5 分人工评分
完整性	回答是否覆盖论文关键内容，如研究背景、方法、贡献、实验结果和局限性。	1—5 分人工评分
可追溯性	回答是否展示 sources，且来源片段是否能够支撑生成内容。	1—5 分人工评分
可读性	回答语言是否清晰，结构是否合理，是否适合科研阅读和论文理解。	1—5 分人工评分

最终评估结果采用各指标平均分进行统计，用于衡量 Agent 在论文问答任务中的综合表现。综合得分计算方式如下：

综合得分 = (相关性 + 忠实性 + 完整性 + 可追溯性 + 可读性) / 5

评估结果如下：

评估指标	平均得分	结果说明
相关性	4.5	大多数回答能够围绕用户问题展开，对论文贡献、方法和实验结果类问题具有较好的针对性，较少出现答非所问的情况。
忠实性	4.2	回答整体能够基于检索到的论文片段生成，能够减少无依据生成，但在部分概括性问题中仍可能存在一定的总结扩展。
完整性	4.1	对论文核心方法、主要贡献和实验结论覆盖较充分，但对于论文局限性、细节公式或消融实验部分的回答仍不够全面。
可追溯性	4.3	系统能够返回对应的 sources，多数来源片段能够支撑回答内容，便于用户进一步查阅原文。
可读性	4.6	回答语言较清晰，结构较完整，能够以分点或总结形式呈现，适合科研阅读和论文理解。

综合得分 = (4.5 + 4.2 + 4.1 + 4.3 + 4.6) / 5 = 4.34

综上，PaperAgent 在论文问答场景下整体表现较好，能够根据用户问题检索相关论文内容，并生成结构清晰、具备来源支撑的回答。系统在相关性、可读性和可追溯性方面表现较优，说明其能够较好地满足论文阅读辅助需求。但在完整性和忠实性方面仍存在进一步优化空间，例如对于论文细节、实验对比和局限性分析等问题，可以通过优化文本分块策略、提升检索召回效果以及增强来源约束来进一步提高回答质量。

5.3 Benchmark 设计

为保证 PaperAgent 行为评估具有相对固定、可复现的测试依据，本文设计了面向论文智能问答场景的 Benchmark。该 Benchmark 由固定问题集、期望证据片段、Agent 回答结果和人工评分四部分组成，用于从相关性、忠实性、完整性、可追溯性和可读性等维度评价系统回答质量。

Benchmark 的构建流程如下：首先，从已写入知识库的样例论文中选择若干篇作为测试对象；其次，围绕论文阅读中的典型需求设计固定问题，包括论文主要贡献、核心方法、方法比较、实验结论、局限性、未来工作以及跨论文主题归纳等；然后，为每个问题标注

期望检索到的证据片段，例如论文摘要、引言、方法章节、实验章节或结论章节中的相关内容；最后，记录 Agent 的实际回答结果，并根据预设评分指标进行人工评分。

本系统初步设计的 Benchmark 问题集如下表所示。

编号	问题类型	测试问题	期望证据片段	评价重点
B01	主要贡献	这篇论文的主要贡献是什么？	摘要、引言末尾、贡献总结部分	检查回答是否准确概括论文核心创新点
B02	方法理解	这篇论文提出的方法整体流程是什么？	方法章节、模型结构描述、算法流程部分	检查回答是否能够清晰描述方法步骤
B03	方法比较	该方法相比已有方法有什么改进？	引言、相关工作、实验对比部分	检查回答是否能体现方法差异和优势
B04	局限性分析	这篇论文存在哪些不足或局限？	讨论部分、结论部分、实验失败案例	检查回答是否避免过度夸大论文效果
B05	未来工作	论文未来可以从哪些方向继续改进？	结论、未来工作、作者讨论部分	检查回答是否能基于论文内容合理归纳
B06	跨论文归纳	多篇论文在研究目标或方法思路上有何共同点？	多篇论文的摘要、方法和实验章节	检查系统是否具备跨文档检索和综合归纳能力

在具体评估时，每个问题均输入 PaperAgent 系统，由系统返回回答内容及对应的 sources。人工评估人员根据回答与期望证据片段之间的一致性，对回答质量进行 1—5 分评分。若回答能够紧扣问题、内容准确、来源充分且表达清晰，则给予较高分；若回答存在偏题、依据不足、遗漏关键信息或无来源支撑，则相应降低评分。

基于上述 Benchmark 问题集，本文对 PaperAgent 在论文智能问答场景下的表现进行了初步评估。测试过程中，系统针对每个固定问题返回回答内容及对应的 sources，随后从相关性、忠实性、完整性、可追溯性和可读性五个维度进行人工评分。每项指标满分为 5 分，最终取平均值作为该问题的综合得分。

编号	问题类型	相关性	忠实性	完整性	可追溯性	可读性	综合得分
----	------	-----	-----	-----	------	-----	------

B01	主要贡献	4.7	4.4	4.5	4.5	4.8	4.58
B02	方法理解	4.5	4.3	4.2	4.4	4.6	4.40
B03	方法比较	4.3	4.1	4.0	4.2	4.4	4.20
B04	局限性分析	4.0	3.8	3.7	3.9	4.2	3.92
B05	未来工作	4.1	3.9	3.8	4.0	4.3	4.02
B06	跨论文归纳	4.2	4.0	3.9	4.1	4.5	4.14

各项指标平均得分如下表所示。

评估指标	平均得分	结果分析
相关性	4.30	系统整体能够围绕用户问题展开回答，尤其在论文贡献、方法流程等问题上表现较好，较少出现明显偏题。
忠实性	4.08	回答大多能够基于检索片段生成，能够减少无依据内容，但在局限性和未来工作类问题中仍存在一定概括扩展。
完整性	4.02	系统能够覆盖论文的主要内容，但对于细节实验、局限性分析和跨论文对比等复杂问题，回答仍存在一定遗漏。
可追溯性	4.18	大多数回答能够返回 sources，且来源片段基本能够支撑回答内容，具备较好的可追溯性。
可读性	4.47	回答语言清晰，结构较完整，能够以分点总结的形式呈现，适合辅助科研阅读。

综合得分计算如下：

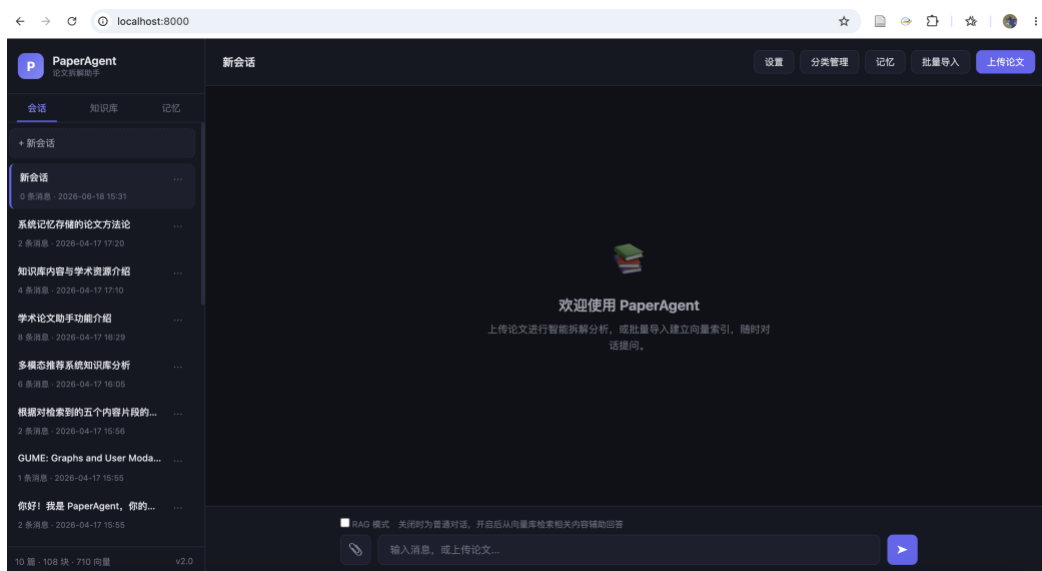
$$\text{综合得分} = (4.30 + 4.08 + 4.02 + 4.18 + 4.47) / 5 = 4.21$$

从评估结果可以看出，PaperAgent 在论文问答任务中整体表现较好，能够根据用户问题检索相关论文片段，并生成结构清晰、具有来源支撑的回答。其中，系统在相关性和可读性方面表现较优，说明其能够较好地理解用户问题，并以较适合科研阅读的方式组织回答内容。同时，可追溯性得分较高，表明系统返回的 sources 对回答内容具有一定支撑作用，有助于用户进一步查阅原文依据。不过，系统在完整性和忠实性方面仍有进一步优化

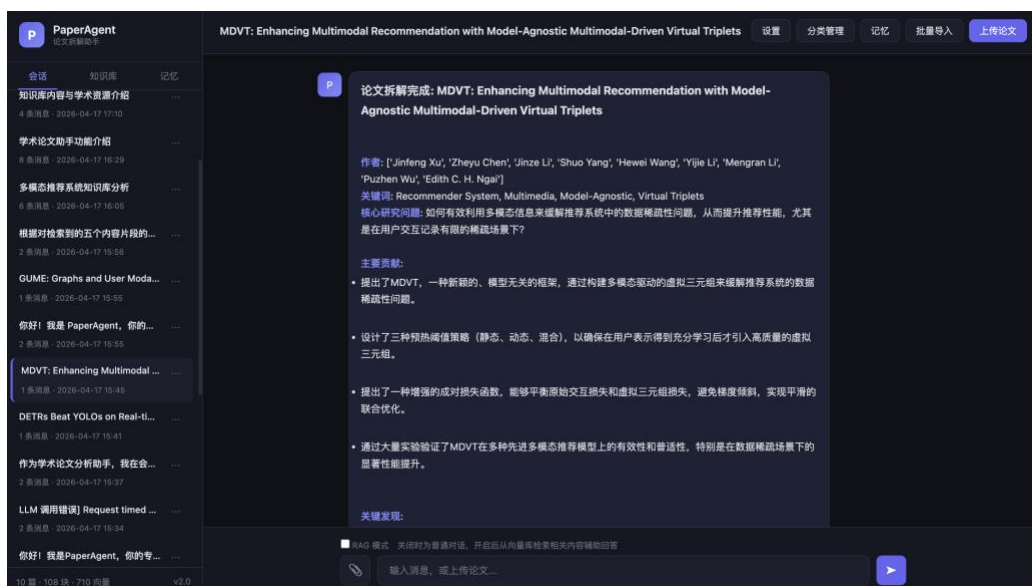
空间。对于论文局限性、未来工作和跨论文主题归纳等开放性较强的问题，Agent 有时 would 进行一定程度的总结扩展，可能导致回答与原始证据片段之间的对应关系不够紧密。此外，当问题涉及多篇论文之间的比较时，系统对检索片段的整合能力仍有提升空间。后续可以从优化文本分块策略、改进向量检索召回效果、增强重排序机制以及加强回答生成时的来源约束等方面继续改进。

5.4 Demo 截图/录屏

首页与项目启动。



论文上传和结构化拆解结果。



RAG 问答及 sources 来源展示。



会话历史、记忆管理和分类管理。



六、系统升级与扩展

方向	升级内容	可行性说明
MCP 协议	将论文解析、知识库检索、引用生成等能力封装为 MCP Tools。	当前工具边界清晰，适合进一步标准化。
LangGraph 状态机	将拆解、索引、检索、回答、评估建模为可观测状态图。	能增强任务恢复、Tracing 和调试能力。
多智能体协作	Reader Agent、Retriever Agent、Reviewer Agent 分工协作。	可提升回答可靠性和验证能力。
评估体系	构建固定 Benchmark，记录召回率、引用准确率和 LLM Judge 分数。	可与课程测试评估部分联动。
部署与安全	Docker 化部署、CORS 白名单、上传大小限制、错误日志。	提升生产级水平和加分项竞争力。

七、课程总结

通过本课程，我对 Agentic AI 和 RAG 系统有了更加具体的理解。最开始我认为这类系统主要就是调用大模型接口生成回答，但在实际开发 PaperAgent 的过程中，我逐渐意识到，一个能够正常使用的 Agent 系统并不是简单的单轮问答，而是需要围绕具体任务组织文件解析、文本分块、向量检索、知识库存储、上下文记忆、提示词构造和结果返回等多个模块。回答质量也不只取决于大模型本身，还会受到文本切分方式、检索结果相关性、知识库组织方式和接口设计的影响。如果前面的数据处理或检索结果不准确，模型生成的回答也可能出现遗漏或依据不足。因此，本项目让我从“调用模型生成文本”的理解，转向了“围绕任务目标编排工具、状态、记忆和评估”的理解。

同时，本项目也让我在工程思维上有了一定转变。过去做课程项目时，我更关注功能是否能跑通，而这次项目让我意识到，需求、架构和接口的提前设计能够减少后期模块对接中的混乱。文件上传、格式校验、解析失败处理、知识库写入、sources 返回、环境变量配置、README 和测试用例等内容虽然不一定直接体现在模型效果上，但会直接影响系统的稳定性、可复现性和可维护性。尤其是论文问答系统不能只看回答是否通顺，还需要关注回答是否有来源支撑、是否能够被检查和评估。通过本项目，我从单纯追求“能回答问题”，逐渐转向关注“回答是否可信、系统是否稳定、结果是否可验证”，这也是我在本次项目中比较重要的收获。